

SMART Dust Bin: Revolutionizing Waste Management with AI & IoT

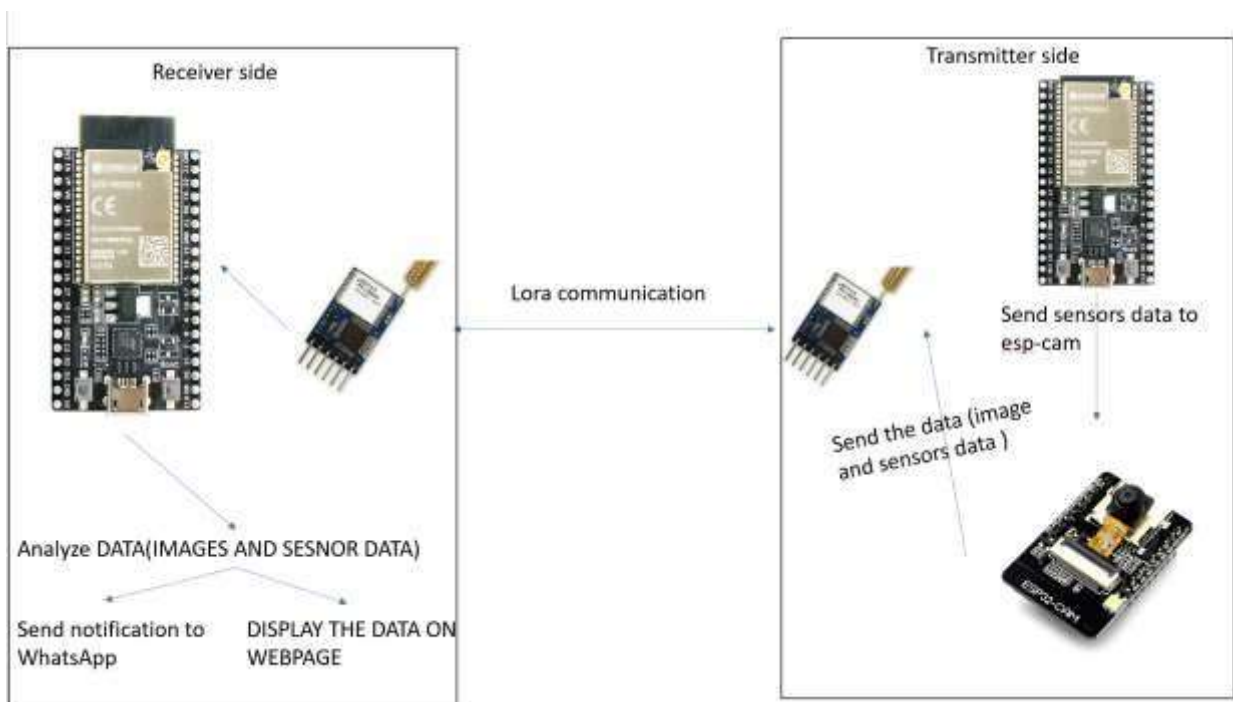
Description:

The **Smart Dust Bin** uses IoT technology and AI-driven image processing to optimize waste management. The system provides real-time updates on bin levels and surrounding cleanliness via a web application. By utilizing **LoraWAN** for wireless communication and AI for image analysis, the system improves efficiency and promotes sustainable waste management practices.

System Architecture

The system architecture consists of:

- **2 ESP32 Dev Boards**
- **ESP32-CAM**
- **2 Lora Modules**
- **Ultrasonic Sensor**
- **AI Model for Image Processing**
- **Web Dashboard for Real-time Monitoring**



Transmitter Side

1. **Bin Level Detection:** An ultrasonic sensor measures the bin's level of fullness.
2. **Image Capture:** The ESP32-CAM captures images of the surrounding area to assess cleanliness
3. **Image Analysis:** The AI model processes the image to detect objects and evaluate cleanliness, determining if the area needs cleaning.

Esp32

Esp-cam

```

07/07/13.276 -> Sending Sleep
07/07/13.276 -> Last Packet Read Status: Delivery Success
07/07/13.380 -> Done with success
07/07/13.380 -> Going to sleep now
07/07/13.483 ->
07/07/13.529 -> *****Sleep*****
07/07/13.585 -> Sending Send Data
07/07/13.741 -> Last Packet Read Status: Delivery Success
07/07/13.712 -> Done With success
07/07/13.975 -> Received message: sleep, Status: sleep
07/07/13.926 -> AT+GPRS=1,37,Rin level: 476 modelPredictor: clean
07/07/13.996 -> Sending Sleep
07/07/13.928 -> Last Packet Read Status: Delivery Success
07/07/13.986 -> Done with success
07/07/13.986 -> Going to sleep now
07/07/14.000 -> nls_sendmsg: n_sendmsg: sendmsg: not_drvinfo: not_drvinfo: ep_sendmsg
07/07/14.000 -> mode:000, sleep done
07/07/14.000 -> load:0x00000000, len:4312
07/07/14.000 -> load:0x00000000, len:16480
07/07/14.000 -> load:0x00000400, len:4
07/07/14.000 -> load:0x00000404, len:1584
07/07/14.000 -> entry: 0x00000000
07/07/14.070 -> Received message: Send Data
07/07/14.192 -> Image captured successfully.
07/07/14.225 -> HSI Image saved successfully in SD card , and passed to the model
07/07/14.262 -> The new ip: clean, and the model: score: 0.947969
07/07/14.274 -> Sent with success
07/07/14.378 -> Last Packet Read Status: Delivery Success
07/07/14.512 -> Received message: Sleep
07/07/14.578 -> Going to sleep now

```

4. **Data Organization & Transmission:** The bin level data and model prediction are combined with the id of the bin and then transmitted via **LoRaWAN** to the receiver.

Receiver Side

1. **Data Separation:** The system separates the received bin level data and model prediction and the bin ID.
2. **Bin Level Alerts:** Bin level data is sent to WhatsApp and displayed on the web dashboard for monitoring.

[illegible]

Design By: Therapist Myo

Web Application Features

Dashboard Overview

- **Real-Time Monitoring:** Displays current bin level, cleanliness status, and the latest images.
- **Historical Data Visualization:** Graphical representation of bin levels over the past 10 days.
- **Multi-Bin Support:** Enables users to monitor multiple bins or locations from one dashboard.

AI Model

Object Detection and Classification

- **Feature:** Detects cleanliness and classifies objects around the bin.

Hardware Setup

Transmitter Setup

ESP32-CAM Setup

1. Upload the AI Model and Image Capture Code:

- **Model:** Upload the `tflite.ino` to the ESP32-CAM. This code contains the TFLite model that performs object detection and sends the prediction to the ESP32 transmitter using ESP-NOW.
- **Image Capture:** To capture and save images to a PC, run the `capture_image.py` script while connected to the ESP32-CAM via an FTDI chip. The image is transferred over Serial.

ESP32 Transmitter Setup

1. Ultrasonic Sensor Pins:

- Bin 1:
 - Trigger Pin: GPIO 12
 - Echo Pin: GPIO 13
- Bin 2:
 - Trigger Pin: GPIO 2
 - Echo Pin: GPIO 15
- (Feel free to modify the pins as needed.)

2. LoRa Module Pins:

- RX: GPIO 22
- TX: GPIO 23

3. Upload the Code:

- Upload `LoRa_Transmitter_updated.ino` to the ESP32 transmitter.

Receiver Setup

1. LoRa Module Pins:

- RX: GPIO 22
- TX: GPIO 23

2. Modify Wi-Fi and WhatsApp Credentials:

- Update the Wi-Fi credentials, API key, and phone number for WhatsApp notifications in the code `LoRa_Receiver.ino` before uploading.

3. Upload the Code:

- Upload `LoRa_Receiver.ino` to the ESP32 receiver.

4. Web Application:

- Open the web app on your phone or PC, ensuring it is connected to the same Wi-Fi network as the ESP32 receiver.

Testing the System

Once everything is set up, you can test the model. The ESP32-CAM will capture images, run the object detection model, and send the data to the web app. You can view real-time predictions, bin levels, and cleanliness status on the web app.

Retrain the AI Model

Follow these steps to retrain the AI model for the Smart Bin project:

Step 1: Clone the GitHub Repository

1. **Open Command Prompt (cmd)** on your local PC.
2. **Navigate to the folder** where you want to clone the repository:

```
cd /path/to/your/folder
```

3. **Clone the repository** using the following command:

```
git clone https://github.com/kareem1637/Smart-bin.git
```

Step 2: Open the Project

1. **Open the project folder** using Visual Studio Code (VS Code) or upload the project to **Google Colab**.

Step 3: Install Required Libraries

1. In the terminal (or Colab environment), **install the required libraries** by running:

```
pip install -r requirements.txt
```

Step 4: Focus on Relevant Folders

The project contains many folders, but for the AI model, you only need the following:

- **source code:** Contains the code for training and testing the model.
- **dataset:** Contains images for training and testing the model.
- **models:** Contains the last version of the trained model.

Step 5: Train and Test the Model

1. Navigate to the **source code** folder.
2. Open the `baseModel.ipynb` file. This is where the model is built, trained, and tested.
 - The file reads the dataset, compiles, trains the model, and then saves the trained model in the same folder as `baseModel.ipynb`.
3. After training, **move the saved model file** from the `source code` folder to the `models` folder for future use.

Step 6: Update the Dataset (If Needed)

1. If you need to retrain the model, **update the dataset:**
 - Navigate to the `dataset/train` folder.
 - Inside, you will find two subfolders: `clean` and `dirty`.
 - Add new images to these respective subfolders (`clean` for clean images, `dirty` for dirty images).

Step 7: Retrain the Model

1. Once the dataset is updated, run the code in `baseModel.ipynb` again.
2. The model will be retrained using the new images in the dataset.
3. At the end of the training process, you will see performance results such as accuracy and the confusion matrix.

Model Testing, Conversion to TF Lite, and Deployment

Step 1: Test the Base Model

1. **Open `test_baseModel.ipynb`** in the **source code** folder.
2. This notebook:
 - Loads the trained model.
 - Loads test images from the `pred` folder.
 - Runs predictions and displays the results.
3. **Add Your Test Images:**
 - You can add your own test images by placing them in the `pred` folder.
4. **Modify the Model Path:**

- Before running the notebook, change the model path to the location of the newly saved model:

```
model = load_model('replace with the path of the new saved model')
```

5. **Run the Code** to test the model on the images in the `pred` folder. The predictions will be displayed.

Step 2: Convert the Model to TF Lite

1. **Open `TF_liteModel.ipynb`** located in the **source code** folder.
2. **Read the Saved Model:**
 - This notebook reads the trained model (saved in the `models` folder) and converts it into a TensorFlow Lite (TFLite) model.
 - Ensure that the correct model is loaded by modifying the path to the model in the notebook:

```
model = load_model('replace with the path of the new saved model')
```

3. **Convert the Model:**
 - The model will be converted to TFLite format and saved in the `models` folder.
 - **Rename the Converted Model:** After conversion, rename the TFLite model to avoid conflicts with previous versions:

```
converter.save('models/new_model_name.tflite')
```

Step 3: Test the TF Lite Model

The last three blocks of code in `TF_liteModel.ipynb` are used to test the TFLite model:

1. *Testing on the Test Images:*

- This block tests the TFLite model on the test images to ensure that its accuracy is consistent with the base model.
- Ensure the test images are in the appropriate folder, and run the block.

2. *Testing on Captured Images:*

- This block reads images from the `captured_image` folder and runs predictions on them.
- You can add or remove images, but **keep at least five images in the folder** to avoid any errors during testing.

3. Testing on a Single Image:

- In this block, you can test the TFLite model on a single image.
- Change the image path to the desired image:

```
image_path = 'path/to/your/image'
```

Run the code to check the model's prediction on the selected image.

These steps will guide you through testing the base model, converting it to TF Lite format, and performing various tests on the TFLite model to ensure that it works as expected before deploying it on the ESP32.

Deploy the Model on ESP32-CAM

Follow these steps to convert your TensorFlow Lite model to C++ format and deploy it on the ESP32-CAM.

Step 1: Convert the TFLite Model to C++ Format

1. **Open a New Google Colab Notebook:**
 - Start a new notebook in Google Colab.
2. **Upload Your TFLite Model:**
 - Upload the TFLite version of your final model (`TF_LITE_FinalModel2.tflite`) to the notebook.
3. **Install `xxd` (if not already installed):** Run the following command to ensure `xxd` is available:

```
!apt-get install -y xxd
```

4. **Convert the Model:** Run the following command to convert your TFLite model into C++ format:

```
!xxd -i TF_LITE_FinalModel.tflite > trash_classifier.cc
```

5. **Download the Output File:**
 - After running the above command, download the `trash_classifier.cc` file to your local device.

Step 2: Prepare the C++ Header File

1. **Open the `trash_classifier.cc` File:**
 - Open the downloaded `trash_classifier.cc` file in a text editor.

2. Copy the Model Data:

- Locate the curly brackets { in the file and copy all the content from the curly bracket to the end of the file.
- Paste this content into file named `TF_LITE_MODEL.h`.

3. Maintain the Array Name: Ensure the variables names remains the same as before:

```
alignas(8) const unsigned char TF_LITE_MODEL_data[];  
unsigned int TF_LITE_MODEL_data_len;
```

Step 3: Upload the New Model to the ESP32-CAM

1. Upload the New Header File:

- Upload the `TF_LITE_MODEL.h` file to your ESP32-CAM project.

2. No Additional Changes Needed:

- You don't need to change anything else in your code. The new trained model is now ready to be used.